

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Chap. 9 Arithmetic instructions - with integer numbers

The FPU instructions covered in this chapter perform arithmetic operations with the value in the TOP data register ST(0) and integer numbers located in memory.

The arithmetic instructions covered in this document are (in alphabetical order):

FIADD	ADD an Integer located in memory to ST(0)
FIDIV	DIVide ST(0) by an Integer located in memory
FIDIVR	DIVide an Integer located in memory by ST(0)
FIMUL	MULTiply ST(0) by an Integer located in memory
FISUB	SUBtract an Integer located in memory from ST(0)
FISUBR	SUBtract ST(0) from an Integer located in memory

FIADD (Add an integer located in memory to ST(0))

Syntax: **fiadd Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Overflow, Precision

This instruction performs a signed addition of the source (*Src*) integer value and the value of ST(0) and overwrites the content of ST(0) with the result. The source must be the memory address of a 16-bit WORD or a 32-bit DWORD integer value (see Chap.2 for [addressing modes of integers](#)). (The [FADD](#) instruction must be used to add a REAL number to the value in ST(0)).

Note that a QWORD integer in memory cannot be added directly to ST(0). If such an addition becomes necessary, the QWORD value must first be loaded to the FPU and then added with the FADD or [FADDP](#) instruction.

An Invalid operation exception is detected if the TOP data register ST(0) is empty or is a [NAN](#), setting the related flag in the [Status Word](#). The [INDEFINITE](#) value would then be inserted in ST(0). (A value of [INFINITY](#) in ST(0) will be treated as a valid number and yield an INFINITY result without any exception being detected.)

A Stack Fault exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A Denormal exception is detected when the content of ST(0) is a [denormalized](#) number, setting the related flag in the Status Word. The addition would still yield a valid result.

A Precision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An Overflow exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of ST(0).

Examples of use:

```
fiadd dword_var      ;add the dword_var integer variable to ST(0)
fiadd word ptr [eax] ;add the WORD value pointed to by EAX to ST(0)
```

If an integer in one of the CPU registers, or an immediate integer value, needs to be added to ST(0), it can be transferred to the stack and used from there. The following code is suggested for adding an immediate value of 100 to ST(0).

```
pushd 100 ;push the value of 100 on the stack
fiadd dword ptr[esp] ;add the DWORD located at memory address [esp] to ST(0)
fxxxx      ;another FPU instruction to insure that the addition instruction
            ;is completed before removing the integer value from the stack
pop reg    ;pop any CPU register which can be trashed to clean the stack
            ;or use that reserved space to store a DWORD value from the FPU
            ;for retrieval into one of the CPU registers or even into memory
            ;or the stack pointer can be restored with add esp,4
```

FISUB (Subtract an integer located in memory from ST(0))

Syntax: **fisub Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Overflow, Precision

This instruction performs a signed subtraction of the source (*Src*) integer value from the value of ST(0) and overwrites the content of ST(0) with the result. The source must be the memory address of a 16-bit WORD or a 32-bit DWORD integer value (see Chap.2 for [addressing modes of integers](#)). (The [FSUB](#) instruction must be used to subtract a REAL number from the value in ST(0)).

Note that a QWORD integer in memory cannot be subtracted directly from ST(0). If such a subtraction becomes necessary, the QWORD value must first be loaded to the FPU and then subtracted with the FSUB or [FSUBP](#) instruction.

An Invalid operation exception is detected if the TOP data register ST(0) is empty or is a [NAN](#), setting the related flag in the [Status Word](#). The [INDEFINITE](#) value would then be inserted in ST(0). (A value of [INFINITY](#) in ST(0) will be treated as a valid number and yield an INFINITY result without any exception being detected.)

A Stack Fault exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A Denormal exception is detected when the content of ST(0) is a [denormalized](#) number, setting the related flag in the Status Word. The subtraction would still yield a valid result.

A Precision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An Overflow exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of ST(0).

Examples of use:

```
fisub word_var      ;subtract the word_var integer variable from ST(0)
fisub dword ptr[esi+8] ;subtract the DWORD pointed to by [ESI+8] from ST(0)
```

If an integer in one of the CPU registers, or an immediate integer value, needs to be subtracted from ST(0), it can be transferred to the stack and used from there. The following code is suggested for subtracting the content of EAX from ST(0).

```
push eax ;push the content of EAX on the stack
fisub dword ptr[esp] ;subtract the DWORD at memory address [esp] from ST(0)
fxxxx      ;another FPU instruction to insure that the subtraction
            ;is completed before removing the integer value from the stack
pop eax    ;or pop any CPU register which can be trashed to clean the stack
```

;or use that reserved space to store a DWORD value from the FPU
;for retrieval into one of the CPU registers or even into memory

FISUBR (Reverse subtraction of ST(0) from an integer located in memory)

Syntax: **fisubr Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value,
 Overflow, Precision

This instruction performs a signed subtraction of the value of ST(0) from the source (*Src*) integer value and overwrites the content of ST(0) with the result; the value of the source remains unchanged. The source must be the memory address of a 16-bit WORD or a 32-bit DWORD integer value (see Chap.2 for [addressing modes of integers](#)). (The [FSUBR](#) instruction must be used to subtract the value in ST(0) from a REAL number).

Note that ST(0) cannot be subtracted directly from a QWORD integer in memory. If such a subtraction becomes necessary, the QWORD value must first be loaded to the FPU and then the subtraction performed with the FSUBR or [FSUBRP](#) instruction.

An Invalid operation exception is detected if the TOP data register ST(0) is empty or is a [NAN](#), setting the related flag in the [Status Word](#). The [INDEFINITE](#) value would then be inserted in ST(0). (A value of [INFINITY](#) in ST(0) will be treated as a valid number and yield an INFINITY result without any exception being detected.)

A Stack Fault exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A Denormal exception is detected when the content of ST(0) is a [denormalized](#) number, setting the related flag in the Status Word. The subtraction would still yield a valid result.

A Precision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An Overflow exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of ST(0).

Examples of use:

```
fisubr word_var      ;subtract ST(0) from the word_var integer variable
                      ;and replace the content of ST(0) with the result
fisubr dword ptr[esp] ;subtract ST(0) from the DWORD pointed to by [ESP]
                      ;and replace the content of ST(0) with the result
                      ;(see the FISUB example for a typical use with [ESP])
```

FIMUL (Multiply ST(0) by an integer located in memory)

Syntax: **fimul Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value,
 Overflow, Precision

This instruction performs a signed multiplication of the content of ST(0) by the source (*Src*) integer value and overwrites the content of ST(0) with the result. The source must be the memory address of a 16-bit WORD or a 32-bit DWORD integer value (see Chap.2 for [addressing modes of integers](#)). (The [FMUL](#) instruction must be used to multiply the value in ST(0) by a REAL number).

Note that ST(0) cannot be multiplied directly by a QWORD integer in memory. If such a multiplication becomes necessary, the QWORD value must first be loaded to the FPU and then multiplied with the FMUL or [FMULP](#) instruction.

An Invalid operation exception is detected if the TOP data register ST(0) is empty, or is a [NAN](#), or its value is [INFINITY](#) and the source integer has a value of zero, setting the related flag in the [Status Word](#). The

[INDEFINITE](#) value would then be inserted in ST(0). (If the source integer has a non-zero value, a value of INFINITY in ST(0) will be treated as a valid number and yield an INFINITY result without any exception being detected.)

A **Stack Fault** exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A **Denormal** exception is detected when the content of ST(0) is a [denormalized](#) number, setting the related flag in the Status Word. The multiplication would still yield a valid result.

A **Precision** exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **Overflow** exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of ST(0).

Examples of use:

```
fimul dword_var ;multiply ST(0) by the dword_var integer variable
fimul dword ptr[ebp+12] ;multiply ST(0) by the DWORD pointed to by [EBP+12]
                        ;this would be typical code generated by the assembler
                        ;for multiplying ST(0) by a parameter passed to a procedure
```

FIDIV (Divide ST(0) by an integer located in memory)

Syntax: **fidiv Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Underflow, Precision, Zero divide

This instruction performs a signed division of the content of ST(0) by the source (*Src*) integer value and overwrites the content of ST(0) with the result. The source must be the memory address of a 16-bit WORD or a 32-bit DWORD integer value (see Chap.2 for [addressing modes of integers](#)). (The [FDIV](#) instruction must be used to divide the value in ST(0) by a REAL number).

Note that ST(0) cannot be divided directly by a QWORD integer in memory. If such a division becomes necessary, the QWORD value must first be loaded to the FPU and then the division performed with the FDIV or [FDIVP](#) instruction.

An **Invalid operation** exception is detected if the TOP data register ST(0) is empty, or is a [NAN](#), or if the value of both the source integer and ST(0) is zero, setting the related flag in the [Status Word](#). The [INDEFINITE](#) value would then be inserted in ST(0). (A value of [INFINITY](#) in ST(0) will be treated as a valid number and yield an INFINITY result without any exception being detected, even if the source integer has a value of zero.)

A **Stack Fault** exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A **Denormal** exception is detected when the content of ST(0) is a [denormalized](#) number or the result is a denormalized number, setting the related flag in the Status Word. The division would still yield a valid result.

A **Precision** exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **Underflow** exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word.

A **Zero divide** exception will be detected if the source integer has a value of zero, and the value of INFINITY will overwrite the content of ST(0), unless the value of ST(0) is also zero which would be an invalid operation as indicated above. (This exception is not detected if the value of ST(0) is already INFINITY.)

Examples of use:

```

fidiv word_var      ;divide ST(0) by the word_var integer variable
fidiv dword ptr [ebp-8] ;divide ST(0) by the DWORD pointed to by [EBP-8]
                        ;this would be typical code generated by the assembler
                        ;for dividing ST(0) by a LOCAL dword integer variable

```

FIDIVR (Reverse division of an integer located in memory by ST(0))

Syntax: **fidivr Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Overflow, Precision, Zero divide

This instruction performs a signed division of the source (*Src*) integer value by the content of ST(0) and overwrites the content of ST(0) with the result; the value of the source remains unchanged. The source must be the memory address of a 16-bit WORD or a 32-bit DWORD integer value (see Chap.2 for [addressing modes of integers](#)). (The [FDIVR](#) instruction must be used to divide a REAL number by the value in ST(0)).

Note that a QWORD integer in memory cannot be divided directly by ST(0). If such a division becomes necessary, the QWORD value must first be loaded to the FPU and then the division performed with the FDIVR or [FDIVRP](#) instruction.

An Invalid operation exception is detected if the TOP data register ST(0) is empty, or is a [NAN](#), or if the value of both the source integer and ST(0) is zero, setting the related flag in the [Status Word](#). The [INDEFINITE](#) value would then be inserted in ST(0). (A value of [INFINITY](#) in ST(0) will be treated as a valid number and yield a zero result without any exception being detected.)

A Stack Fault exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A Denormal exception is detected when the content of ST(0) is a [denormalized](#) number, setting the related flag in the Status Word. The division would yield a value of INFINITY, unless the source integer is zero which would yield a value of zero.

A Precision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An Overflow exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of ST(0).

A Zero divide exception will be detected if ST(0) has a value of zero and the value of INFINITY will overwrite the content of ST(0), unless the integer value is also zero which would be an invalid operation as indicated above.

Examples of use:

```

fidivr dword_var    ;divide the dword_var integer variable by ST(0)
                        ;and replace the content of ST(0) with the result
fidivr word ptr [esi+ebx] ;divide the WORD located at [ESI+EBX] by ST(0)
                        ;and replace the content of ST(0) with the result

```

[RETURN TO](#)
[SIMPLY FPU](#)
[CONTENTS](#)